

TRENTOS-M RPi_SPI_SD

- [Resources](#)
 - [Driver Code](#)
 - [Other resources](#)
- [Overview](#)
- [Storage Interface implementation](#)
- [Limitation](#)
- [Hardware](#)
 - [Raspberry Pi SPI interfaces](#)
 - [MicroSD Card reader interface](#)
- [Usage](#)
 - [CMake](#)
 - [CMakeES](#)
- [Platforms](#)

Resources

Driver Code

- Driver for BCM2837 SPI peripheral access: <https://www.airspayce.com/mikem/bcm2835/index.html>
- Driver for SDFilesystem access: <https://os.mbed.com/components/SD-Card/>

Other resources

- http://www.interfacebus.com/MicroSD_Card_Pinout.html
- <https://openlabpro.com/guide/interfacing-microcontrollers-with-sd-card/>
- <https://docs-emea.rs-online.com/webdocs/1111/0900766b811118fa.pdf>
- http://elm-chan.org/docs/mmc/mmc_e.html
- <http://pierre.baudu.in/other/clustersize.en.html>
- <https://support.microsoft.com/en-us/help/140365/default-cluster-size-for-ntfs-fat-and-exfat>

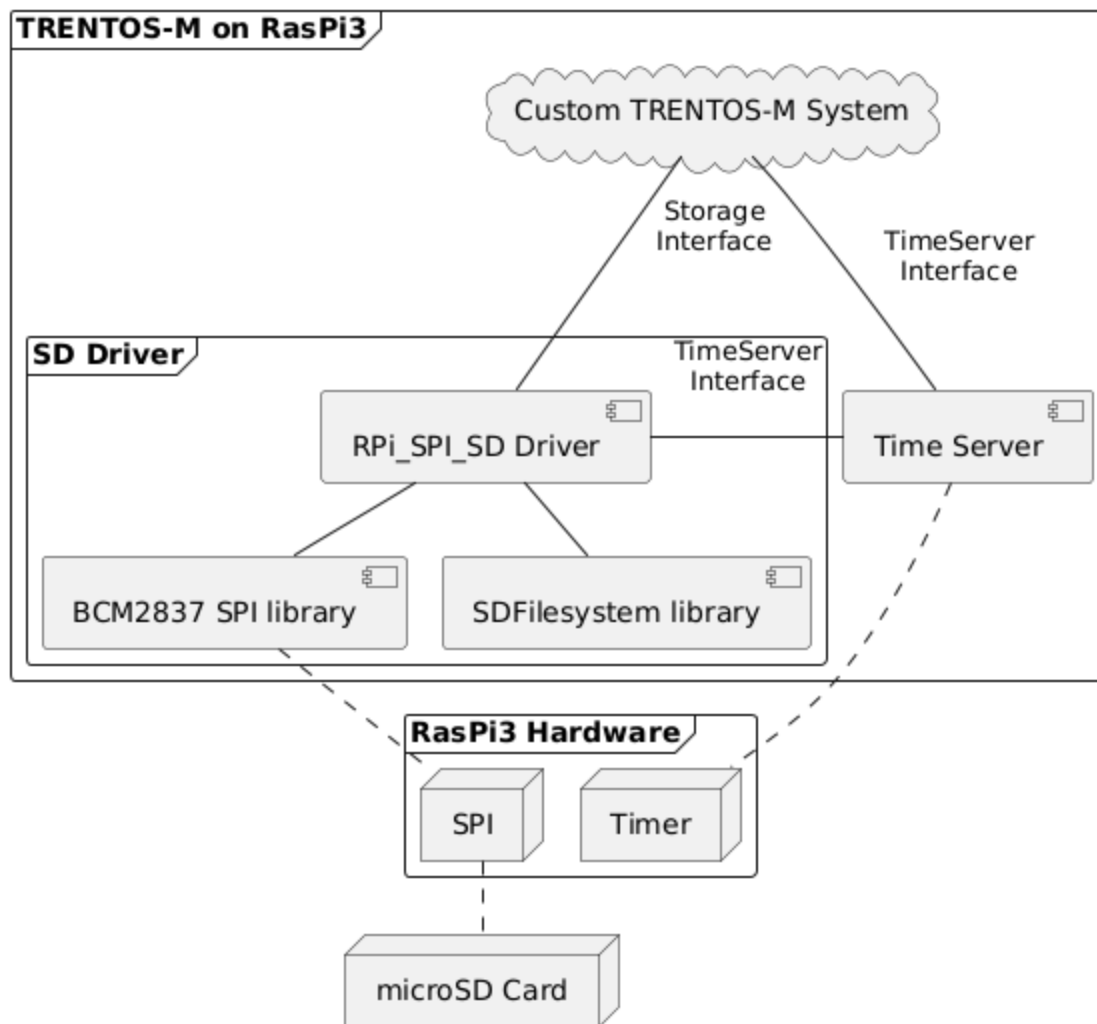
Overview

The RPi_SPI_SD component is basically a combination of a redesigned version of the RPi_SPI_Flash component and a 3rd party SD C++ library from mbedOS. A micro-SD card is connected to the Raspberry Pi over a microSD card reader allows communication with Raspberry Pi over SPI.

A 3rd party library is used in the driver for BCM2837 SPI peripheral access and microSD card access.

The driver (*RPi_SPI_SD.c*) provides the TRENTOS-M Storage interface on top, which can be used directly by applications. Furthermore, the driver links SPI calls from the SDFilesystem library to the BCM2837 library by implementing some callback functions (*impl_spisd_spi_transfer()*, *impl_spisd_spi_cs()*, *impl_spisd_wait()*). SPI functions in the SDFilesystem library are linked to these functions in the *post_init()* function.

The SDFilesystem library needs to implement waits, thus timer functionality needs to be provided which makes it necessary to implement a TimeServer component.



RPi_SPI_SD Driver - Architecture

Storage Interface implementation

As you write blockwise to SD cards, the SD Card driver implements the Storage Interface in the following way:

- *storage_rpc_write()*: read first block that should be written, modify partially and write entire block back. Then overwrite entire blocks for as long as entire blocks can be filled with new data. The last block to be written must be read again, modified partially and wrote back to the SD card.
- *storage_rpc_read()*: read first block that should be read and only copy actually wanted data to the dataport. Then read entire blocks for as long as entire blocks can be read. The last block that has data to be read will only be partially read.
- *storage_rpc_erase()*: read first block that should be erased, modify partially with 0xFF bytes and write entire block back. Then overwrite entire blocks with 0xFF for as long as entire blocks can be filled with new data. The last block to be erased must be read again, filled partially with 0xFF and wrote back to the SD card.
- *storage_rpc_getSize()*: calls *disk_capacity()* function from SDFilesystem library.
- *storage_rpc_getState()*: calls *disk_status()* function from SDFilesystem library.

Internally, the block size is fixed to 512 Bytes.

Limitation

Currently the SD driver component is tested for raw data operations (read, write, erase) and for filesystems FATFS (and LITTLEFS also tested but not implemented in the filesystem demo). The current SD card driver implementation does not work for SPIFFS. Maybe the SPIFFS configuration is not correctly set (can be found in *seos_tests/seos_sandbox/libs/os_filesystem/src/lib/SpiffFs.c*).

Hardware

Raspberry Pi SPI interfaces

General information regarding SPI usage on the RPi can be found in the official RPi SPI documentation: <https://www.raspberrypi.org/documentation/hardware/raspberrypi/spi/README.md>

In order to be able to use SPI, a potential device has to be connected and wired to the respective Pins accordingly. See also <https://www.raspberrypi.org/documentation/usage/gpio> or <https://de.pinout.xyz/pinout/spi> for the assignment. Note that the numbering on the 2x20 GPIO pin headers is different to the BCM2738 SoC's internal I/O numbering. The documentation often refers to the SoC's internal I/O signals when describing the I/O functions, while for actually connecting wires and peripherals physically, referring to the 2x20 GPIO headers is more practical.

Signal		RasPi GPIO 2x20 Pin Header	BCM2837 SoC internal I/O
SPI0	MOSI	19	10
	MISO	21	9
	SCLK	23	11
	CE0	24	8
	CE1	26	7
SPI1	MOSI	38	20
	MISO	35	19
	SCLK	40	21
	CE0	12	18
	CE1	11	17
	CE2	36	16

MicroSD Card reader interface

In order to use a SPI-based microSD card reader, the respective device has to be wired to the RPi first. Exemplarily, the respective mapping for a microSD card reader connected at SPI0 is shown in the table below:

microSD-Card reader				RPI	
PIN	Color	SD-Name	SPI-Name	PIN	RPI-Name
1	brown	DAT2	-	-	-
2	yellow	CD/DAT3	CS	24	CS
3	green	CMD	DI	19	MOSI
4	orange	VDD	VDD	17	Voltage (3.3V!!!)
5	violet	CLK	SCLK	23	SCLK
6	black	VSS	VSS	25	Ground
7	blue	DAT0	DO	21	MISO
8	grey	DAT1	-	-	-

Attention: SD cards are operated at 3.3 voltage level.

Better compare these values again with help of the recommended connection (http://www.interfacebus.com/MicroSD_Card_Pinout.html) and Raspberry Pi pinout (<https://www.raspberrypi.org/documentation/usage/gpio/>).

Another device that should work is a microSD card sniffer from Adafruit. The connection mapping is shown in the table below:

microSD-Card sniffer Adafruit			RPI	
PIN	SD-Name	SPI-Name	PIN	RPI-Name
1	DAT2	-	-	-
2	CD/DAT3	CS	24	CE0
3	CMD	DI	19	MOSI
4	GND	GND	25	GND
5	VCC	Supply Voltage	17	Voltage 3.3V!!!
6	CLK	SCLK	23	SCLK
7	DAT0	DO	21	MISO

8	DAT1	-	-	-
---	------	---	---	---

The following image shows the microSD-card reader connected to the Raspberry Pi:

TODO: take picture and upload it

Usage

CMake

RPi_SPI_SD Driver - Usage 1

```
DeclareCAMESCComponent_RPI_SPI_MSD(
  <TypeNameSD>
  LIBS
    <TimeServerClient>
)
```

CAMECS

RPi_SPI_SD Driver - Usage 2

```
#include "RPI_SPI_SD/RPI_SPI_SD.camkes";
DECLARE_COMPONENT_RPI_SPI_MSD_DRV(<TypeNameSD>)

#include "TimeServer/camkes/TimeServer.camkes"
DECLARE_COMPONENT_TimeServer(<TypeTimeServer>)

assembly {
  composition {
    // MSD
    DECLARE_AND_CONNECT_INSTANCE_RPI_SPI_MSD(<TypeNameSD>, <InstanceNameSD>)

    // MSD_Storage_tester
    component <TypeMyApplication> <InstanceMyApplication>;

    connection sel4RPCCall      tester_msd_storage      (from <InstanceMyApplication.rpc>, to
<InstanceNameSD.rpc>);
    connection sel4SharedData    tester_msd_storage_port(from <InstanceMyApplication.dataport>, to
<InstanceNameSD.dataport>);

    //-----
    // TimeServer
    //-----
    DECLARE_AND_CONNECT_INSTANCE_TimeServer(
      TimeServer,
      timeServer,
      <InstanceNameSD.timerServer_rpc>, <InstanceNameSD.timeServer_notify>
    )
  }

  configuration {
    CONFIGURE_INSTANCE_RPI_SPI_MSD(<InstanceNameSD>)
  }
}
```

Platforms

- RPi3
- RPi4 (branch: rpi4_porting)